Beyond the Sort Order: Hierarchy Intervals and Vector Indexes in a B+ Tree Ontology Store

Lam Nguyen
Microsoft
USA
lamng3.work@gmail.com

Abstract

An ontology store is asked two kinds of question. *Structural* ones, such as "which instances have a type at or below class C", have an exact answer in the data. *Semantic* ones, such as "which terms are closest in meaning to t", have no answer in the data and need a similarity measure. Sorted indexes are the workhorse of triple stores, and it is tempting to think that sorted means close. We show it does not. A bandwidth argument proves that no linear order of points in two or more dimensions keeps all neighbors within a short scan, and on the Gene Ontology with sentence embeddings the best order we tried, the postorder of the class hierarchy, finds only 38% of a term's ten nearest neighbors within 100 positions. We then use the order for what it can do. A postorder numbering of the class DAG with an interval set per class, kept in the engine's own B+ tree, turns "everything under C" into a few range scans. Against materializing the RDFS closure, it stores 8.1× fewer triples and a 4.2× smaller file on synthetic instances over the Gene Ontology, and answers within 1.8× of its page reads; against rewriting the query it reads 48× fewer pages for large classes. For similarity we add vector indexes inside the same engine: exact search, IVF with posting lists stored as B+ tree key ranges, product quantization, and HNSW. Pre-filtering through the intervals is exact and 10^2–10^4× faster than any index strategy for classes below about 1 000 members; beyond 5 000, a masked HNSW search wins. The filter is correlated with the vectors: for a query outside a small class, post-filtering IVF finds 7% of the right answers, because the neighbors of a term tend to lie under the term's own ancestors, not under the class asked for. We implement everything in OntoDB, an open-source C++ ontology database, and every number in this paper regenerates from one script.

1 Introduction

Ontologies are stored and queried as graphs of triples, and the engines that hold them index those triples in sorted structures: B+ trees over permutations of subject, predicate, and object [14]. A sorted index answers a bound prefix with one range scan, and this is the right tool for the questions it was built for. Two newer questions are not among them.

The first asks about the hierarchy. A query for the instances of `Material` should return the instances of `Metal` too, because `Metal` is a subclass. The second asks about meaning: given a term, which other terms are close to it? This is the question behind retrieval over ontologies, term matching, and ontology alignment, and it is what a vector index answers. The two questions meet in the third, "the terms most similar to t that lie under C".

This paper asks what a B+ tree ontology store should do about them, and it makes four contributions.

(1) **Order is not similarity.** We prove that any linear order of a d-dimensional point set separates some neighbors by $\Omega(n^{(d-1)/d}/d)$ positions (Lemma 1), and measure how much locality the orders available to a store do give on a real ontology with real embeddings (Section 4). A hierarchy order is far better than first-seen ids and still leaves most neighbors out of a short scan.
(2) **Hierarchies are intervals.** We label the class DAG so that the descendants of a class are a few integer intervals, bound the number of intervals by the number of non-tree edges (Lemma 3), and keep them in a B+ tree beside the triples. On the Gene Ontology the mean class needs 2.0 intervals; against materialization it saves 8.1× the triples, and against rewriting it saves 48× the page reads (Section 5).
(3) **Vector indexes in the engine.** We add exact, IVF, PQ, and HNSW search to the engine, and store IVF posting lists as key ranges of the same B+ tree that holds the triples (Section 6).
(4) **Filtering by a class.** We compare pre-filtering, post-filtering, and masked search for "nearest under C", and show that in an ontology the filter is correlated with the vectors, which breaks the independence assumption behind the usual cost estimate (Section 7).

Everything is built in OntoDB, an open-source ontology database in C++20 (Section 8); the benchmark is one script.

2 Background and Related Work

Triple stores and inference. RDF stores keep triples in several sorted permutations; OntoDB keeps three (SPO, POS, OSP), enough to make every bound/unbound pattern a prefix scan. RDFS entailment is usually either *rewritten* into the query or *materialized* at write time, as RDFox does [14], with maintenance by counting or by delete-and-rederive [7]. Section 5 adds a third option that stores no entailed triple.

Labelling a hierarchy. Agrawal, Borgida, and Jagadish [1] store a compressed transitive closure by numbering a spanning forest in postorder and giving each node a set of intervals; Grust [5] numbers XML trees in pre- and postorder so axes become regions of a plane. We use the former, on class DAGs, inside a B+ tree store, and measure it against the other two ways to answer a type query.

Vector search. Approximate nearest neighbor search has three families that matter here: inverted files and product quantization [10], proximity graphs, of which HNSW [13] is the standard, and disk-oriented graphs such as DiskANN [9]. FAISS [11] packages the first two. Pan, Wang, and Li survey vector database systems and the hybrid queries (attribute plus vector) that are hard for them [18].

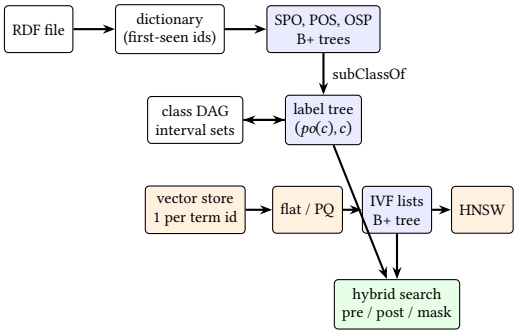


Figure 1: What OntoDB holds. Blue boxes are B+ trees of the same kind; the class DAG is rebuilt from the `subClassOf` triples, and vectors come from an external embedding model.

ACORN [19] makes HNSW traverse only the nodes that pass a predicate.

Graph and RDF systems with vectors. Neo4j offers an HNSW vector index queried from Cypher [15]; Amazon Neptune Analytics offers vector similarity over graph nodes [2]; Fluree provides inline similarity functions and HNSW indexes for RDF [4]; GraphDB has had similarity indexes queried from SPARQL since version 8 [17]. In the documentation we read, none turns the class hierarchy into range-scannable intervals, in the engine that holds the triples, to restrict a vector search.

Embeddings of hierarchies. Euclidean embeddings of names are what we use; hyperbolic embeddings [16] represent hierarchies more faithfully and would change the vectors, not the indexes.

Adaptive radix trees. The ART [12] orders keys by byte, so its range scan is lexical; DuckDB persists it [8]. We discuss it in Section 9.

3 System Overview

Figure 1 shows what OntoDB holds. The triples are dictionary-encoded and indexed in three B+ tree permutations, as before. The class hierarchy is read from the `rdfs:subClassOf` triples, labelled, and kept in a fourth B+ tree. Vectors, one per term id, sit in a store beside them, with three optional indexes; the IVF index is again a B+ tree. A hybrid query touches the label tree and one vector index, never the triple trees.

4 What a Sorted Index Can Say About Meaning

The order of OntoDB's trees. A triple is a 24-byte key of three dictionary ids compared with `memcmp`. The dictionary assigns ids densely, in first-seen order, and the loader interns the terms of each statement in file order. So the order of every tree is the order in which terms first appeared in the input. Two classes are adjacent in the index only if they were adjacent in the file.

No order holds similarity.

Table 1: Share of the 10 nearest neighbors (by cosine) found within $\pm W$ positions of a term, and the median position gap to the nearest neighbor. GO, 38 092 terms, 2 000 queries.

Order	gap	±50	±100	±500	±1000	±5000
dictionary id (first seen)	1739	.195	.211	.258	.284	.438
hierarchy, deepest parent	45	.328	.377	.498	.561	.735
hierarchy, first parent	37	.319	.366	.481	.546	.726
label, alphabetical	1749	.310	.334	.379	.408	.558
random	10823	.003	.005	.024	.049	.242

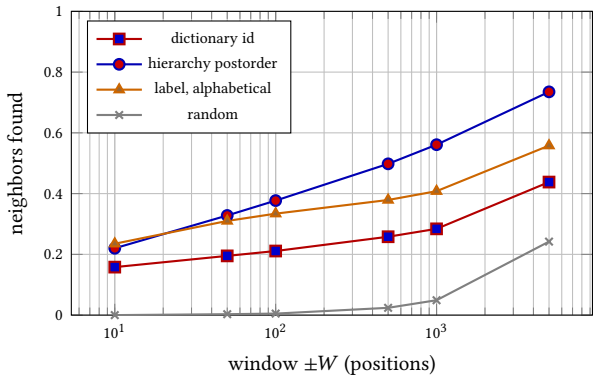


Figure 2: The data of Table 1: neighbors found in a window of $\pm W$ positions, by order.

Lemma 1 (Bandwidth). *Let G have n vertices and diameter D. In every linear order of the vertices some edge joins vertices at least $(n-1)/D$ positions apart.*

Proof. The first and last vertices of the order are $n-1$ apart and joined by a path of at most D edges; some edge of the path covers at least $(n-1)/D$. □

For the neighbor graph of a d-dimensional grid with $n = k^d$ points (diameter $d(k-1)$) this gives $(k^d-1)/(d(k-1)) \geq k^{d-1}/d$ positions, in any order. A scan of width w cannot contain all neighbors unless w is that large. Space-filling curves keep neighbors close on average and need several scans to cover a query ball; they do not escape the lemma.

How much locality do real orders give? We took the Gene Ontology (release 2026-07-26, without obsolete terms: 38 092 classes, 57 456 `is_a` edges), embedded each term's name with `all-MiniLM-L6-v2` [20] (384 dimensions, on a CPU, 45 s), and for 2 000 random terms took the ten nearest by cosine. For each order of the terms, Table 1 gives the share of those neighbors within $\pm W$ positions.

Three things follow. First, the hierarchy's postorder is the best order, and it is much better than OntoDB's first-seen ids: the nearest neighbor is 45 positions away instead of 1 739. Second, even so, a scan of 100 positions finds 38% of the ten neighbors, and a window of ±1000 positions (2.6% of the ontology) still misses 44%; Lemma 1 is not an artifact of the grid. Third, first-seen ids are not random: the Gene Ontology's ids were minted in related batches, so ids carry some locality (0.21 at ±100) that depends on the file

Algorithm 1: Interval sets for a class DAG

Input: edges (c, p): c is a subclass of p
Output: $po(c)$ and an interval set $I(c)$ for each class

collapse strongly connected components (Tarjan), number them parents first;
for *each component c, parents first* **do**
  $depth(c) \leftarrow 1 + \max_p\ depth(p)$; `// 0 for a root`
  $tp(c) \leftarrow$ the parent with the largest depth;
number the forest $\{(c, tp(c))\}$ in postorder; record $lo(c)$;
for *each component c, children first* **do**
  $I(c) \leftarrow \textsc{Merge}\big(\{[lo(c), po(c)]\} \cup \bigcup_{c \to v} I(v)\big)$; `// sort, join overlapping or adjacent`

and not on the engine. Of the ten neighbors, 12% are a parent or child and 32% are within two `is_a` steps; a two-step neighborhood holds 45 terms (0.12% of the ontology), so hierarchy proximity is highly informative about meaning without being sufficient.

The order is therefore useful for structure and not a substitute for a vector index.

5 Hierarchies as Intervals

Labelling. Let the class hierarchy be a DAG, after collapsing each cycle ($A \sqsubseteq B \sqsubseteq A$) into one class, since such classes are equivalent. Choose a spanning forest by taking, for each class with several parents, the parent with the longest path from a root. Number the forest's nodes $1..n$ in postorder, and let $lo(c)$ be the smallest number in the tree subtree of c. For a forest, d is at or below c iff $lo(c) \le po(d) \le po(c)$. For a DAG each class carries a *set* of intervals, built bottom-up (Algorithm 1).

Lemma 2 (Correctness). *d is at or below c iff $po(d)$ lies in an interval of $I(c)$.*

Proof. Induction over the order in which Algorithm 1 builds I: the numbers covered by $I(c)$ are exactly those of the descendants of c. □

Lemma 3 (Size). *Let $E'(c)$ be the non-tree edges with tail at or below c. Then $|I(c)| \le 1 + |E'(c)|$.*

Proof. A descendant d is reached from c by a path. If it uses no non-tree edge, d is in the tree subtree of c; otherwise let v be the head of the last non-tree edge on it, and d is in the tree subtree of v. So the covered set is $\text{own}(c) \cup \bigcup_{e \in E'(c)} \text{own}(\text{head}(e))$, a union of $1 + |E'(c)|$ intervals. □

The labelling takes $O(n + m)$ plus the merges. A tree needs one interval per class. We check Lemma 3 on every class of the Gene Ontology and on 60 random DAGs in the test suite.

Index. The labels go into B+ trees of OntoDB: key $(po(c), c, 0)$ for the classes, and $(po(t), x, 0)$ for each triple $(x, type, t)$. The descendants of c, or the instances of c and its subclasses, are one range scan per interval of c. An instance insert adds one key. Labels are rebuilt when the hierarchy changes (here in 15 ms for GO); incremental maintenance with gaps [1] is not built. Dictionary ids are untouched: ids appear in three permutations, the log, and snapshots, and renumbering them to follow the hierarchy would rewrite all of it.

Table 2: Interval labels for the Gene Ontology.

Spanning-forest rule	non-tree edges	intervals	mean / p99 / max	build
deepest parent	19 367	76 570	2.01 / 17 / 741	14.9 ms
first parent	19 367	87 769	2.30 / 22 / 740	14.4 ms

Table 3: Descendants of a class in GO, cold 64-frame pool, mean over up to 40 classes per row.

Below C	method	scans	page reads	μs
10–99	intervals	5.9	5.8	59
	one scan per class	30.6	16.3	149
100–999	intervals	29.1	13.2	147
	one scan per class	268.2	65.7	803
1000+	intervals	101.3	40.1	528
	one scan per class	3303.6	694.1	8998

5.1 Evaluation

Setup. Gene Ontology as above; LUBM(1) [6] with its ontology (43 classes, 103 410 triples); and 200 000 synthetic entities, one to three types each drawn uniformly from GO's classes (495 552 triples), because GO has no instances. Release build on an Apple M2 Pro, one thread. Page reads are counted on a cold 64-frame buffer pool, the methodology of OntoDB's indexing benchmark.

Label size. Table 2 shows that GO needs 76 570 intervals for 38 092 classes, 2.01 per class, 598 KiB in all; the closure it replaces has 393 448 ancestor pairs, 9 MiB at 24 bytes each and one permutation. The spanning-forest rule matters little (deepest parent saves 13%). The tail is long: one class has 741 intervals, bounded by Lemma 3. Labels build in 15 ms. Asking whether d is below c takes 27 ns against 775 ns for a search up the parents.

Descendants of a class. Table 3 compares the B^+-tree interval scans with a breadth-first search that scans POS once per class. For a class with a thousand or more descendants the intervals need 101 scans instead of 3 304, 40 page reads instead of 694, and 0.5 ms instead of 9 ms.

Instances of a class. Table 4 compares the three strategies on the synthetic instances. Materializing the closure turns 495 552 triples into 4 017 973 (8.1×), the file from 63.6 to 309.5 MiB, and the load from 5.1 s to 180 s. The interval index adds 9.3 MiB (15%) and 0.17 s. On queries, the interval index is close to the materialized store and far from rewriting: for a class with a thousand or more descendants it reads 265 pages against 182 (materialized) and 12 803 (rewrite), and runs in 4.6 ms against 3.8 and 82 ms. All three return identical answers on every sampled class. On LUBM(1) the hierarchy is shallow (43 classes) and the three are within a factor of about two on every query; materialization adds 15% to the triples. The benefit of intervals grows with the depth and breadth of the hierarchy, and LUBM has little of either.

Table 4: Instances of C on 200 000 synthetic entities over GO, mean over 30 classes per row (cold 64-frame pool). Top: cost of the store.

Strategy	triples	file (MiB)	load (s)
rewrite	495 552	63.6	5.09
intervals	495 552	72.9	5.26
materialize	4 017 973	309.5	180.31

Below C	strategy	answers	page reads	μs
10–99	intervals	278	8.4	90
	rewrite	278	87.5	571
	materialized	278	6.6	444
100–999	intervals	2 954	40.9	1 461
	rewrite	2 954	1 153	7 280
	materialized	2 954	22.5	987
1000+	intervals	29 904	264.8	4 555
	rewrite	29 904	12 803	82 441
	materialized	29 904	182.0	3 776

6 Vector Indexes in the Engine

The engine keeps one unit-length float vector per term id, in memory, loaded from a checksummed file whose records name terms by spelling, so a file survives a dictionary that numbers terms differently. Embeddings come from outside; OntoDB stores and searches them. Four index families are available, each a different trade between exactness, memory, and build time.

Flat. Score every vector: $O(nd)$ and exact. With 8-bit scalar quantization (one scale per vector, $s = \max|x|/127$) each coordinate is off by at most $s/2$, so for a unit query a score is off by at most $(s/2)\sqrt{d}$, in a quarter of the memory.

IVF.. Spherical k-means with k-means++ seeding [3] gives L centroids; each vector goes to its nearest. A query scores the centroids and reads the p nearest lists. The cost is $Ld + p(n/L)d$, minimal at $L = \sqrt{pn}$. The design choice is where the lists live: *as key ranges of the B+ tree that holds the triples.* The pair (list, row) is a key (list, row, 0), so a list is a prefix scan, an insert is a tree insert, and the lists page, cache, and latch like every other index of the engine. Under a filter the search keeps probing the nearest remaining lists until it holds k allowed rows, so a filter that excludes everything near the query costs more instead of returning nothing.

Product quantization. Cut a vector into m pieces and replace each by the index of its nearest of 256 learned centers [10]: m bytes per vector. A query builds m tables of 256 dot products and scores a code as the sum of m lookups, which equals the dot product with the decoded vector, so all the error is on the stored side.

HNSW.. The layered proximity graph of Malkov and Yashunin [13], with the paper's neighbor-selection heuristic: a candidate is kept only if it is nearer the new node than to every neighbor already kept. Layer 0 allows $2M$ links. Search takes an optional filter and still crosses nodes it may not return.

Table 5: Vector indexes on GO (recall@10, mean time per query, memory of the structure, build time).

Index	setting	recall	μs	MiB	build (s)
flat float32	exact	1.000	2 238	55.8	—
flat 8-bit	exact over codes	0.996	4 554	14.1	—
flat PQ m=48	codes only	0.778	819	1.7	10.4
flat PQ m=48	re-rank 100	0.998	905	1.7†	10.4
IVF L=512	nprobe 8	0.892	149	56.5	17.5
IVF L=512	nprobe 16	0.929	255	56.5	17.5
IVF L=512	nprobe 64	0.976	928	56.5	17.5
IVF L=512, 8-bit	nprobe 16	0.926	263	14.8	17.5
IVF L=512, PQ 96	nprobe 16	0.839	178	4.2	38.4
HNSW M=16	ef 10	0.993	44	59.7	15.0
HNSW M=16	ef 40	0.997	108	59.7	15.0
HNSW M=32	ef 160	0.999	474	63.4	28.0

† re-ranking reads the float vectors as well.

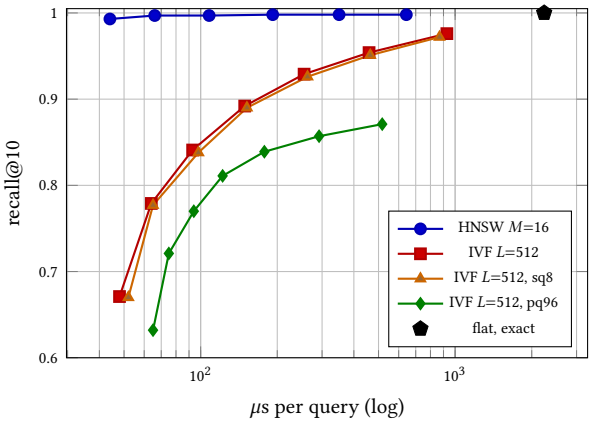


Figure 3: Recall@10 against time per query on GO (38 092 vectors, 384 dimensions).

6.1 Evaluation

38 092 GO terms, 384 dimensions, 1 000 query terms (the term itself excluded from its answer), exact search as ground truth, single thread. Table 5 gives selected points and Figure 3 the recall–latency curves.

A baseline that is not slow by accident. Our first run scored with a plain dot-product loop. A compiler may not vectorize it without `-ffast-math`, and the exact scan took 8.3 ms, which would have made the speed-ups look about twice as large: HNSW at ef=10 is 103 times faster than that baseline and 51 times faster than the vectorized one. With eight independent accumulators the exact scan takes 2.2 ms (13 Gflop/s), and every number here and in Section 7 uses that. We report it because a speed-up over "exact" depends entirely on how fast exact is.

What the numbers say. HNSW dominates on this data: recall 0.993 at 44 μs, 51× faster than the exact scan, for 3.9 MiB of links on top of the vectors. IVF needs 255 μs for 0.929 and 928 μs for 0.976. IVF over the B+ tree is about 2.7× more expensive per scored row

than a contiguous scan (about 157 ns against 59 ns): each entry is a decoded 24-byte key and a random read of the vector. That is the price of lists that page, cache, and update with the triples. The cost model holds in shape: more lists make a query cheaper until the centroid scan dominates (L=2048 pays 124 μs at nprobe 1). Product quantization is for memory, not speed, at this size: codes alone reach 0.78 or 0.88 recall in 1.7 or 3.5 MiB, and re-ranking the best 100 exactly recovers 0.998 and 1.000 but needs the float vectors. 8-bit codes keep 0.996 recall in a quarter of the memory and are slower than float32, because converting bytes to floats costs more than the cache misses they avoid at 56 MiB.

Caveat. The queries are terms of the ontology, whose neighbors are also terms of it. Queries from outside the vocabulary, such as free text, would be harder, and we do not measure them.

7 Filtering a Vector Search by a Class

A hybrid query asks for the k terms nearest t that lie at or under class C. Let $s = |\text{subtree}(C)|/n$. We compare four ways of answering it, all returning terms of the subtree only.

- **Pre-filter.** Resolve C to its members first (one range scan per interval), then score only those, exactly. About $s\,n\,d$ work.
- **Post-filter.** Ask the index for $k' > k$, keep the results under C (a membership test is a binary search over C's intervals; nothing is scanned), and multiply k' by four until k remain or the index is exhausted. If the filter were independent of the similarity, about k/s candidates would be needed.
- **Masked search.** Give the index a predicate, "the row's label lies in an interval of C", so a list scan or a graph walk skips rows outside C. Nothing is resolved in advance: each row the search touches costs one binary search. HNSW still crosses rows it may not return, to keep the graph connected, in the spirit of ACORN [19]; IVF keeps probing lists until it has k allowed rows.
- **Auto.** Pre-filter when C has at most 4 096 classes, otherwise masked HNSW.

The filter is correlated with the vectors. In an ontology the names of a class's descendants are close to the class and to each other, so the neighbors of a term tend to lie under the term's own ancestors. Of the unfiltered top 100 neighbors of a query from *inside* C, 4% to 72% lie under C, rising with the size of C; for a query from *outside* C the share is 0% to 12%. The independence estimate k/s is therefore wrong in both directions: post-filtering is cheap for an inside query and does not terminate usefully for an outside one.

7.1 Evaluation

For 20 classes in each of five subtree-size buckets we ran the ten nearest terms under C for a query that is a member of C ("inside") and one that is not ("outside"), against the exact answer over the members. IVF uses 512 lists and 16 probes, HNSW M=16, ef=64. Table 6 gives recall and time.

Exact pre-filtering wins for small classes. Up to about 1 000 members, resolving the members through the interval index and scoring them exactly takes 3 to 62 μs and is the exact answer. Every index-based strategy is slower, from 1.7× (masked IVF at 101–1 000 members, at recall 0.96) to over 10 000× (post-filtered HNSW on the smallest classes), because few of the vectors an index touches belong to the class. A graph search for a rare filter crosses most of the graph: masked HNSW reads 16 ms for a class of at most 100 members, the failure ACORN is designed to remove.

Post-filtering fails for outside queries. IVF post-filter recall falls to 0.07 on the smallest classes and 0.64 on the largest; HNSW reaches full recall only by searching again and again (37 ms for a class of at most ten members, in six rounds).

Masked HNSW wins for large classes. Above 5 000 members it takes 105 μs inside and 900 μs outside, exact, against 1.4 ms for resolving and scoring the members.

Do not resolve the filter. Our first masked search resolved C by interval scans and built a bitmap per query. For a class of over 5 000 members that took 669 μs, of which about 560 μs was resolving. Testing the label of each row the search touches, one binary search over a handful of intervals, brought the same search to 105 μs. This is where interval labels earn their keep a second time: they make membership an $O(\log |I(C)|)$ test with no scan.

A cost model for the crossover. Let c_m be the cost of pre-filtering per member (an interval scan, a hash lookup from term to row, and one dot product): from Table 6, $c_m \approx 0.13$–0.20 μs for classes of a few hundred members or more. Pre-filtering beats a masked graph search when $|C| < T = c_h/c_m$, where c_h is the masked search's time. For a query inside C, $c_h \approx 105$–183 μs, so $T \approx$ 800–1 000, which matches the observed crossing between the 101–1 000 and 1 001–5 000 columns. For a query outside C the masked walk costs 0.9–4.4 ms on classes of 1 000 or more members, so T is 7 000–25 000. A single threshold has to sit between the two; 4 096 stays below the outside-query crossing and loses at most a factor of 3.8 at the inside-query one.

The auto rule. With a threshold of 4 096 members, auto matches the fastest strategy in every cell but one and is never slower than 0.9 ms. The exception is a class of 1 001–5 000 members with an inside query: 269 μs, against 70 μs for post-filtered HNSW and 183 μs for masked HNSW. Post-filtering would cost 5.5 ms for an outside query on the same class, so the rule gives up a factor of 3.8 in one cell to avoid a factor of 79 in its neighbor.

8 Implementation

OntoDB is a C++20 ontology database: a SPARQL subset over three B+ tree permutations of dictionary-encoded triples, with a buffer pool, a cost-based optimizer, strict two-phase locking, and ARIES recovery, checked by a crash harness. The semantic extension adds about 2 100 lines in `src/semantic`: the class hierarchy and its labels, the label index, the type-closure strategies, the vector store, flat, IVF, PQ, and HNSW indexes, hybrid search, and an engine that drives them from the shell. It adds one B+ tree entry point (a range scan between two keys) and nothing else to the existing engine. 1 000 lines of tests cover it, including property tests that compare every strategy against a naive oracle on random ontologies, and the benchmark is a 1 000-line program, one script. In the shell:

```
\load go.nt          \hindex build
\vload go.ovec       \vindex build hnsw
\similar "apoptotic process" 5 under "cell death"
```

Table 6: Ten nearest terms under a class C, by subtree size, query inside or outside C: recall@10 / μs per query. GO, 38 092 terms. Masked and post-filtered searches use the index; pre-filter resolves the members and scores them exactly.

Strategy	≤10	11–100	101–1000	1001–5000	>5000
query inside C					
pre-filter	1.00 / 3	1.00 / 9	1.00 / 62	1.00 / 292	1.00 / 1 404
post-filter, HNSW	1.00 / 37 553	1.00 / 1 739	1.00 / 427	1.00 / 70	1.00 / 68
post-filter, IVF	0.86 / 903	0.87 / 558	0.93 / 293	0.98 / 198	0.99 / 211
masked HNSW	1.00 / 16 653	1.00 / 15 846	1.00 / 1 150	1.00 / 183	1.00 / 105
masked IVF	1.00 / 1 253	0.95 / 111	0.96 / 104	0.98 / 124	0.99 / 161
auto	1.00 / 3	1.00 / 9	1.00 / 59	1.00 / 269	1.00 / 95
query outside C					
pre-filter	1.00 / 3	1.00 / 9	1.00 / 62	1.00 / 291	1.00 / 1 394
post-filter, HNSW	1.00 / 36 654	1.00 / 27 669	0.99 / 13 607	1.00 / 5 513	1.00 / 836
post-filter, IVF	0.07 / 966	0.02 / 1 009	0.17 / 902	0.33 / 811	0.64 / 591
masked HNSW	1.00 / 16 802	1.00 / 16 611	1.00 / 8 697	1.00 / 4 424	1.00 / 900
masked IVF	1.00 / 1 255	0.79 / 698	0.45 / 286	0.37 / 147	0.66 / 107
auto	1.00 / 3	1.00 / 9	1.00 / 62	1.00 / 295	1.00 / 860

```
 0.865 GO:1902742 apoptotic process involved
       in development             [pre-filter]
 0.802 GO:0060561 apoptotic process involved
       in morphogenesis
\under "programmed cell death" 5
96 classes (4 range scans, 0.012 ms)
```

Vectors are held in memory and the vector indexes are rebuilt, not updated, when the vectors change; labels are rebuilt when the hierarchy changes (15 ms for GO). An engine owns a private buffer pool on a temporary file for the label tree and the IVF lists, so it runs on either of the store's backends.

Reproducing the numbers. Data and models are fetched, not shipped: the Gene Ontology `go-basic.obo` release 2026-07-26 (`scripts/fetch-data go`), embeddings by `all-MiniLM-L6-v2` through `sentence-transformers` on the CPU (`scripts/embed-terms`, 45 s), LUBM(1) and its ontology (`scripts/fetch-data lubm lubm-ontology`). `benchmarks/semantic/run.sh` builds the release binary and writes every table of this paper; seeds are fixed, queries are the same for every method, and the machine is an Apple M2 Pro with 16 GiB, one thread, `-O3`. Run-to-run variation in times is a few percent; page counts and recalls are deterministic.

9 Discussion

Is an ART better? An adaptive radix tree orders keys by byte [12], so its range scan is a lexical range scan, the same one-dimensional order as a B+ tree, and Lemma 1 applies. Giving its leaves sibling links is possible and awkward. Path compression and node growth never move a leaf, but lazy expansion does: inserting into a leaf turns it into an inner node and puts a *copy* of the leaf under it, so a link that pointed at the old object points at an inner node; each insert must also find its sorted neighbors by a partial descent, and concurrent inserts must latch both. The usual alternative is a stack-based iterator, with no links, that climbs and descends between leaves. OntoDB's keys are three fixed-width integers, which leave an ART little to compress, so the B+ tree stays.

Pessimistic or optimistic prefixes. A compressed prefix can be stored in the node (pessimistic) or only its length (optimistic, with the full key checked once at the leaf). Seeking a lower bound needs to know whether a whole subtree is below or above it. With stored bytes the seek decides at the node; with skipped bytes it must load a key from the subtree and compare. The upper bound needs no jumping: a second seek finds the first key past it, and each step compares two keys.

Limits. The vectors in our measurements come from one ontology and one small model that embeds names only; definitions, synonyms, or a domain model would change the vectors, not the indexes. The instance data is synthetic, because GO has none, and LUBM's hierarchy is too shallow to separate the strategies. Everything is single-threaded and in memory. Labels are rebuilt, not maintained under edits to the hierarchy, and only `rdfs:subClassOf` is read; no reasoner. Embeddings are Euclidean; hyperbolic embeddings [16] would represent a hierarchy more faithfully and would change what the vectors say, not how they are searched. Query text needs an embedding model that OntoDB does not link; the shell searches from the vector of a term that is already stored.

What we did not find. We did not find a vector index that makes the hierarchy order unnecessary: neighbors by meaning and descendants are different sets (a third of a term's ten nearest neighbors are within two `is_a` steps, two thirds are not), which is why the two are combined at query time and not at index time.

10 Conclusion

A sorted index orders; it does not say what is close in meaning, and the bandwidth argument and the Gene Ontology measurements agree. What an ontology store can do is use the order for the part of the question that is structural, and a vector index for the rest. Labelling the class DAG with interval sets turns "everything under C" into a few range scans of the engine's own B+ tree, at 2.0 intervals per class on the Gene Ontology and a fraction of the cost of materializing the closure, and it makes membership in a class's subtree a binary search, which is what a filtered vector search needs. Exact

pre-filtering through those intervals is the right answer for classes up to about a thousand members, and a masked graph search for the largest. Both fit in a B+ tree engine, in about two thousand lines.

References

[1] Rakesh Agrawal, Alexander Borgida, and H. V. Jagadish. 1989. Efficient Management of Transitive Relationships in Large Data and Knowledge Bases. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 253–262. https://doi.org/10.1145/67544.66950

[2] Amazon Web Services. 2023. Amazon Neptune Analytics: Vector Similarity Search. AWS documentation and launch announcement. https://aws.amazon.com/blogs/aws/introducing-amazon-neptune-analytics-a-high-performance-graph-analytics, accessed October 2026.

[3] David Arthur and Sergei Vassilvitskii. 2007. k-means++: The Advantages of Careful Seeding. In *Proceedings of the ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 1027–1035.

[4] Fluree PBC. 2026. Fluree Vector Search. Fluree documentation. https://labs.fluree.com/docs/db/indexing-and-search/vector-search, accessed October 2026.

[5] Torsten Grust. 2002. Accelerating XPath Location Steps. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 109–120.

[6] Yuanbo Guo, Zhengxiang Pan, and Jeff Heflin. 2005. LUBM: A Benchmark for OWL Knowledge Base Systems. *Journal of Web Semantics* 3, 2–3 (2005), 158–182.

[7] Ashish Gupta, Inderpal Singh Mumick, and V. S. Subrahmanian. 1993. Maintaining Views Incrementally. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*.

[8] Pedro Holanda. 2022. Persistent Storage of Adaptive Radix Trees (ART) in DuckDB. DuckDB blog. https://duckdb.org/2022/07/27/art-storage.

[9] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: Fast Accurate Billion-Point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[10] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.

[11] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547.

[12] Viktor Leis, Alfons Kemper, and Thomas Neumann. 2013. The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 38–49.

[13] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.

[14] Boris Motik, Yavor Nenov, Robert Piro, Ian Horrocks, and Dan Olteanu. 2014. Parallel Materialisation of Datalog Programs in Centralised, Main-Memory RDF Systems. In *Proceedings of AAAI*.

[15] Neo4j, Inc. 2026. Neo4j Vector Index and Search. Neo4j documentation. https://neo4j.com/developer/genai-ecosystem/vector-search/, accessed October 2026.

[16] Maximilian Nickel and Douwe Kiela. 2017. Poincaré Embeddings for Learning Hierarchical Representations. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[17] Ontotext. 2019. GraphDB 8.9 with Improved Semantic Similarity Search. Ontotext announcement. https://sirma.com/news/graphdb-8-9-with-improved-semantic-similarity-search.html.

[18] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of Vector Database Management Systems. *The VLDB Journal* 33 (2024), 1591–1615.

[19] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 120.

[20] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. In *Proceedings of EMNLP-IJCNLP*.